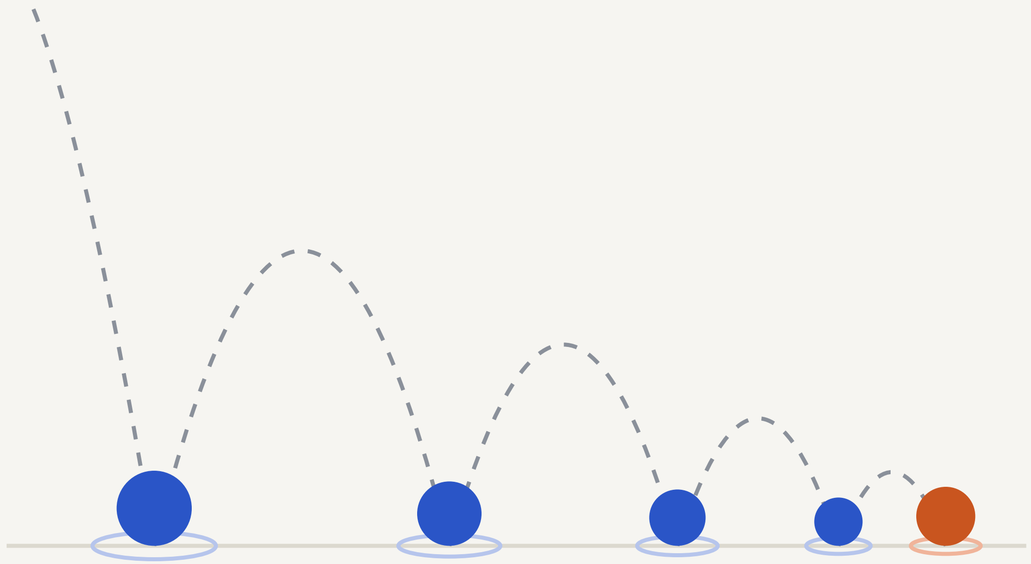
MY CONTENT · MVP

Ricochet.

Système de recommandation d’articles, serverless sur Azure

Darya Filatova · 11/2026



LA MISSION

Quatre livrables attendus

01

Azure Functions

Une première version du système de recommandation.

02

Application

Liste des lecteurs, appel de la Function, cinq articles affichés.

03

GitHub

Les scripts du projet, versionnés.

04

Synthèse

Architecture actuelle, système de recommandation, architecture cible.

Chaque bloc de cette présentation répond à l’un de ces livrables.

LES DONNÉES

Un jeu public, anonymisé, sans notes

2 988 181
clics

Un article n’a ni titre ni texte : un identifiant et 250 nombres.

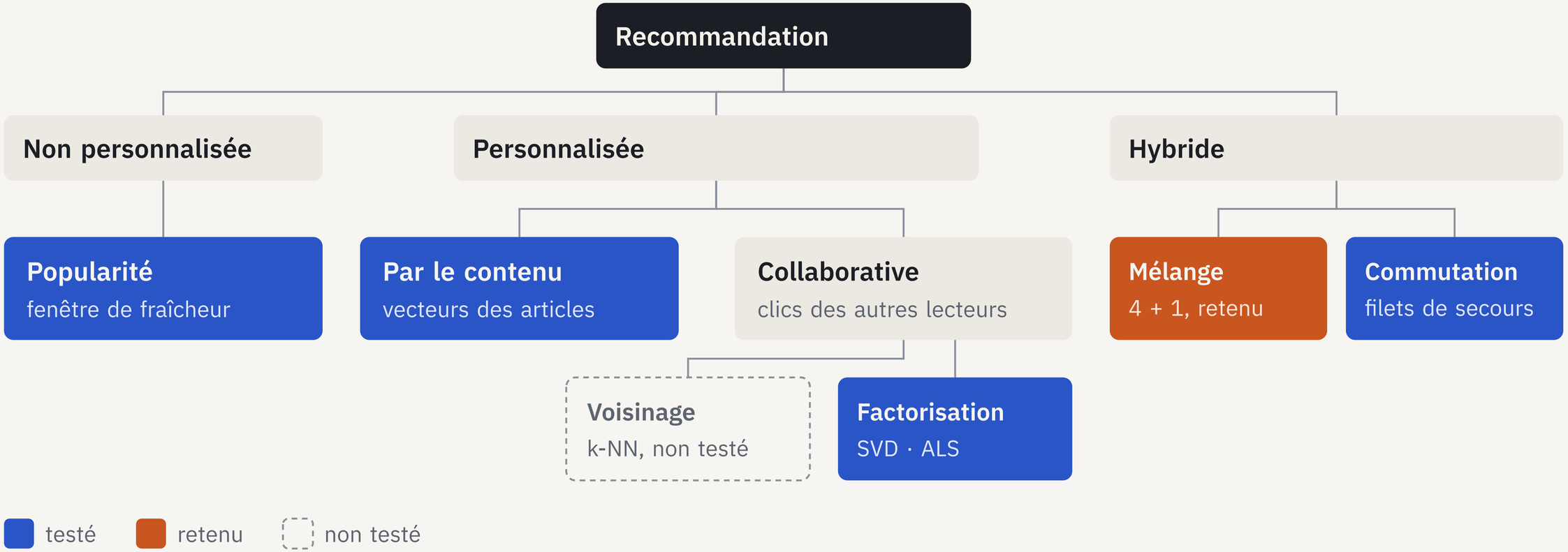
322 897
lecteurs

Pas de notes : seulement des clics.

364 047
articles

L’interface n’affiche que des identifiants.

Quatre familles de méthodes, chacune testée



Comment chaque méthode construit sa liste

Popularité

clics de la dernière heure → comptage, par région

+ Suit l'actualité, sans entraînement

– Ne personnalise rien

Contenu

articles lus → moyenne des vecteurs → les plus proches

+ Marche dès le premier clic, et pour un article nouveau

– Vise mal ; sans limite de date, propose de l'ancien

ALS

clics, cases vides = négatifs → goûts latents

+ Capte des goûts que le contenu ne voit pas

– Muet pour un lecteur ou un article nouveau

SVD

clics en binaire, 4 négatifs → note prédite

+ Rapide à entraîner

– Ne se reproduit pas : 0,0815 en validation, 0,0220 en test

Les méthodes personnalisées butent sur le démarrage à froid : d'où l'hybride.

SYSTÈME DE RECOMMANDATION · LES RÉSULTATS

La popularité bat tous les modèles

Précision · HitRate@5
Le prochain article lu est-il parmi les 5 ?

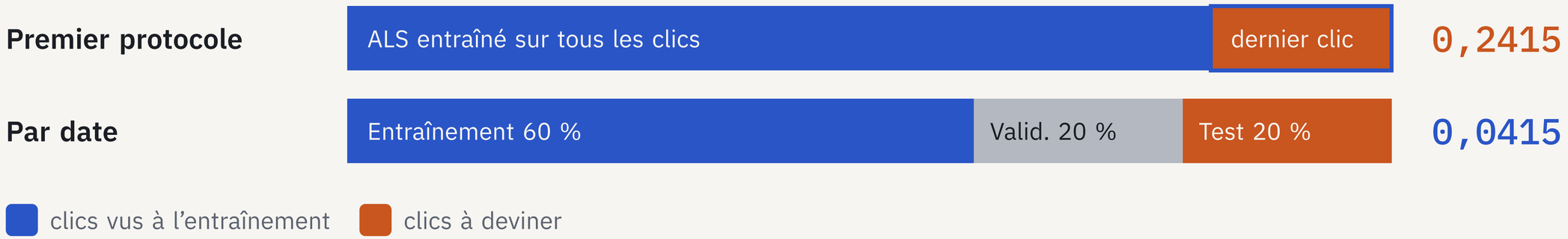
Couverture
Part du catalogue recommandée au moins une fois

Méthode	Précision	Couverture	Articles
Popularité, dernière heure	0,2525	0,003 %	11
ALS	0,0415	0,015 %	≈ 55
SVD	0,0220	0,010 %	≈ 36
Contenu	0,0200	0,27 %	≈ 980
Mixte · retenu	0,2500	0,116 %	≈ 420
Hasard, pour mémoire	0,00001	≈ 100 %	364 047

Le mixte garde la précision de la popularité et expose 38 fois plus d’articles.

COMMENT NOUS AVONS SU QUE LES CHIFFRES ÉTAIENT FAUX

Le modèle avait vu le clic à deviner



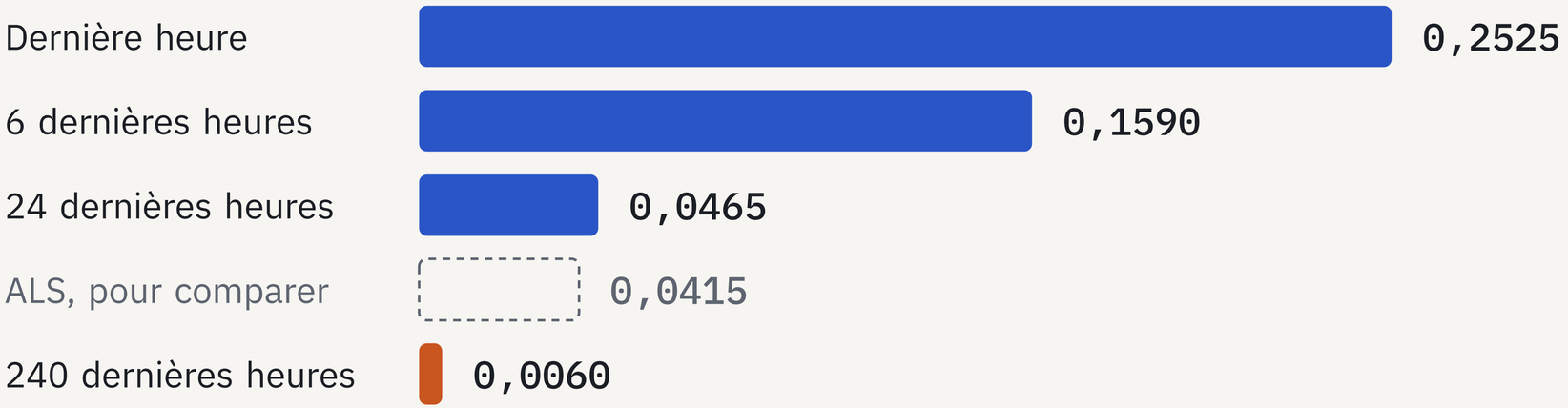
×5,8
de précision imaginaire

D'abord, la cible, le dernier clic de chaque lecteur, faisait partie de l'entraînement. Par date, le test commence après la fin de l'entraînement, et n'est mesuré qu'une fois.

RÉSULTAT PRINCIPAL · LA FRAÎCHEUR

La fraîcheur pèse plus que le modèle

Même méthode partout : proposer les 5 articles les plus lus pendant la fenêtre qui précède. Seule la longueur de la fenêtre change.



×42

entre 1 heure et 240 heures

Pourquoi : **48 %** des articles lus pendant le test n’existaient pas encore à l’entraînement. Un modèle appris sur le passé ne peut pas les proposer.

LE SYSTÈME RETENU

Quatre places au récent, une au contenu

1	2	3	4	5
Le plus lu, dernière heure	Le plus lu, dernière heure	Le plus lu, dernière heure	Le plus lu, dernière heure	Proche des lectures

×38 de catalogue exposé

La place du contenu coûte environ 1 % de précision : le meilleur compromis mesuré entre justesse et couverture.

Jamais de liste vide

Région et fraîcheur, puis fraîcheur seule, puis région seule, puis tout l’historique.

DESCRIPTION FONCTIONNELLE

Un lecteur en entrée, cinq articles en sortie



Paramètre	Rôle
`user_id`	Le lecteur. Seul paramètre obligatoire.
`method`	La stratégie : **mix** par défaut, parmi les cinq testées.
`history`	L'historique d'un lecteur que le service ne connaît pas : rien n'est gardé entre deux appels.
`region`	La région du lecteur : n'agit que pour un lecteur sans historique.
`fresh_only`	Se limiter aux articles récents : activé par défaut.
`n`	Le nombre d'articles renvoyés : 5 par défaut.

Se mettre à jour sans ré-entraîner

Élément	Mise à jour	Quand	Ré-entraînement
Fenêtre de popularité	Recompte des clics de la dernière heure	Toutes les heures	Non : un comptage
Profil d’un lecteur	Moyenne des vecteurs lus	À chaque appel	Non
Nouvel article	Placé par la projection ACP sauvegardée	À sa publication	Non, vérifié
Projection ACP	Recalcul sur le catalogue	Si les thèmes dérivent	Oui, rare et léger
Vecteurs des articles	Un modèle d’embedding du texte	À sa publication	À construire

Seul manque réel pour la production : les 250 nombres d’un article sont fournis par Globo dans le jeu public. My Content devra les produire à partir du texte.

L'APPLICATION · LA FRAÎCHEUR

Une case décochée, le facteur 42 apparaît

FRAÎCHEUR ACTIVÉE · FENÊTRE 1 H

5 article(s) recommandé(s) — mix

1. Article #277107 ★★★★ (9 373 lecteurs) — catégorie 409 · 144 mots · publié le 16/10/2017

2. Article #336254 ★★★★★ (10 627 lecteurs) — catégorie 437 · 170 mots · publié le 16/10/2017

3. Article #157478 ★★★★ (5 888 lecteurs) — catégorie 281 · 259 mots · publié le 16/10/2017

4. Article #352979 ★★★★ (3 784 lecteurs) — catégorie 442 · 169 mots · publié le 16/10/2017

5. Article #284220 ★ (9 lecteurs) — catégorie 412 · 224 mots · publié le 17/10/2017

FRAÎCHEUR DÉSACTIVÉE · TOUT L'HISTORIQUE

5 article(s) recommandé(s) — mix

1. Article #4907 ★★★ (603 lecteurs) — catégorie 2 · 210 mots · publié le 09/10/2017

2. Article #257291 ★★★★ (8 119 lecteurs) — catégorie 391 · 205 mots · publié le 04/10/2017

3. Article #2303 ★★★ (111 lecteurs) — catégorie 1 · 204 mots · publié le 02/10/2017

4. Article #235230 ★★★★ (9 123 lecteurs) — catégorie 375 · 262 mots · publié le 03/10/2017

5. Article #160997 ★ (1 lecteurs) — catégorie 281 · 241 mots · publié le 25/09/2017

À gauche, des articles du 16/10 lus par des milliers de lecteurs. À droite, un article du 25/09 lu par **une** personne.

L'APPLICATION · UN NOUVEAU LECTEUR

Un lecteur inscrit est servi aussitôt

INSCRIPTION · RÉGION OPTIONNELLE



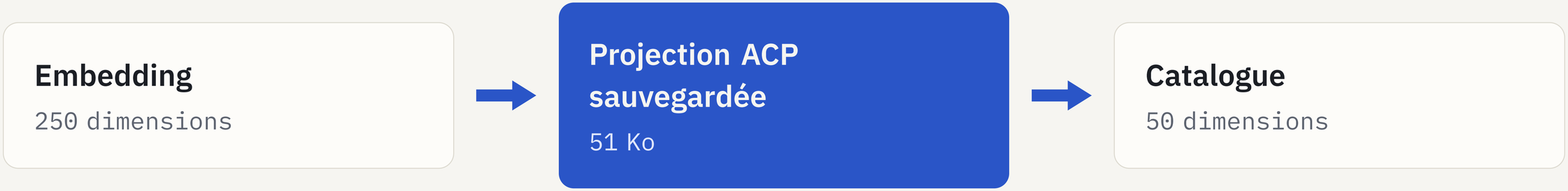
CATALOGUE · 364 047 ARTICLES



À chaque article marqué comme lu, le profil est recalculé à la demande, sans ré-entraînement.

L’APPLICATION · UN NOUVEL ARTICLE

Un nouvel article, recommandé en minutes



Sans réajuster l’ACP

Le nouvel article est placé dans la même base que les 364 047 autres, sans les recalculer.

Vérifié

Trois articles ajoutés sont entrés aussitôt dans le top 5 d’un lecteur proche, sans ré-entraînement.

L'APPLICATION ET LE CODE

Choisir un lecteur, appeler la Function

LE SCÉNARIO DEMANDÉ

1. Liste des user_id
2. Appel HTTP à /api/recommend
3. Cinq articles : identifiant, catégorie, longueur, date

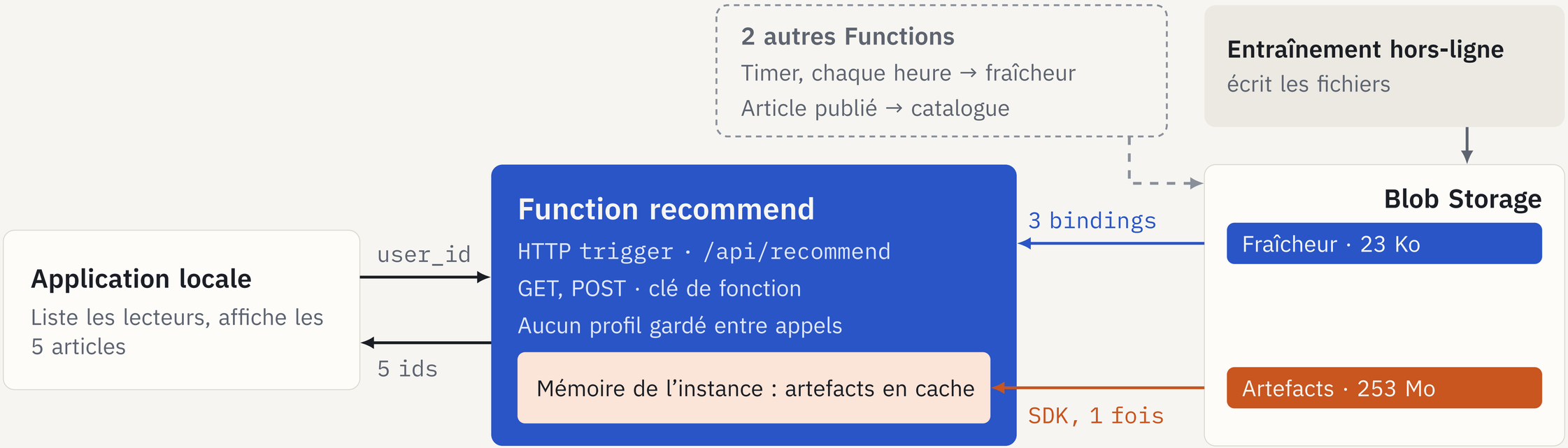
EN PLUS

- Marquer des lectures : le profil est recalculé sans ré-entraînement
- Parcourir le catalogue de 364 047 articles
- Inscrire un lecteur, servi aussitôt
- Changer de stratégie, désactiver la fraîcheur

</> Code : github.com/delnouty/Ricochet-content-recommendation

ARCHITECTURE RETENUE · LA SEULE GRATUITE SANS ÊTRE BRIDÉE

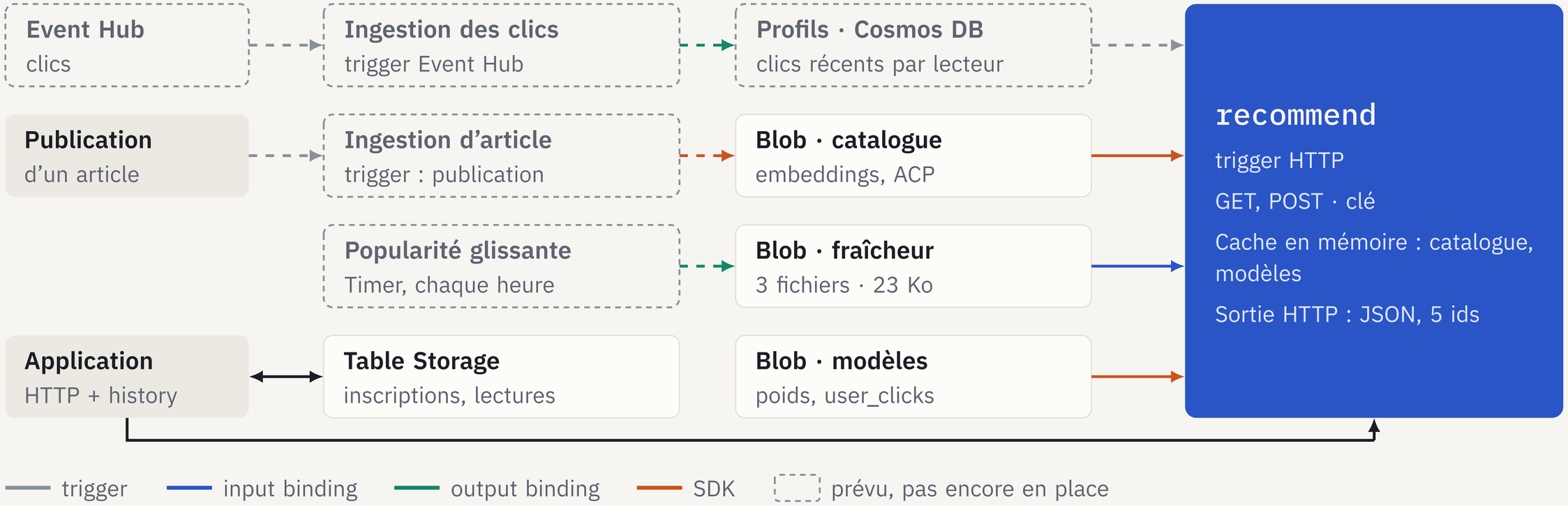
Serverless : une Function et un Blob



Chaque appel HTTP déclenche la Function : elle reçoit les 3 fichiers de fraîcheur par binding et garde les artefacts en mémoire, chargés une seule fois par le SDK. En pointillé, ce qui manque : une Function planifiée (timer, toutes les heures) qui met à jour la fraîcheur, et une Function déclenchée à la publication d'un article qui l'ajoute au catalogue.

ARCHITECTURE CIBLE · FUNCTIONS, TRIGGERS, STOCKAGE

Quatre Functions, un trigger chacune



Aujourd'hui : l'application écrit inscriptions et lectures dans Table Storage et passe l'historique dans la requête ; les articles s'ajoutent par lot avec scripts/add_articles.py. Aucune de ces Functions n'entraîne de modèle.

CONCLUSION

Livré, mesuré, à construire

Livré

- Function recommend sur Azure, Flex Consumption : 0,2 s à chaud
- Application : un lecteur choisi, cinq articles
- Code, CI et 87 tests sur GitHub
- Coût réel : 0 € du 01/08 au 06/11/2026

Mesuré

- La fraîcheur avant le modèle : ×42 entre 1 h et 240 h
- Le mixte : 0,2500, et 38 fois plus de catalogue que la popularité
- Un score faux détecté : ×5,8, corrigé par un découpage par date

À construire

- Fraîcheur recalculée chaque heure par un timer
- Ingestion des articles et des clics par événement
- Ré-entraînement automatique : récupérer les données en CI
- Le vrai juge : le taux de clic sur de vrais lecteurs

ANNEXE · QUESTIONS À DISCUTER · LA CONTRAINTE

Gratuité.

Rester dans les quotas gratuits d’un abonnement Azure payant à l’usage, où tout dépassement est facturé.

Le choix d’architecture était libre. Cette contrainte l’a tranché, et tout le reste en découle.

POURQUOI CETTE ARCHITECTURE

Deux architectures serverless proposées

Architecture 1 · avec API

Application locale

↓

Azure Function
relais vers l'API

↓

API dédiée
héberge le modèle

Deux services à déployer et à maintenir.

Architecture 2 · sans API

Application locale

↓

Azure Function
prédiction intégrée

↑

Blob Storage
fichiers et modèles, input binding

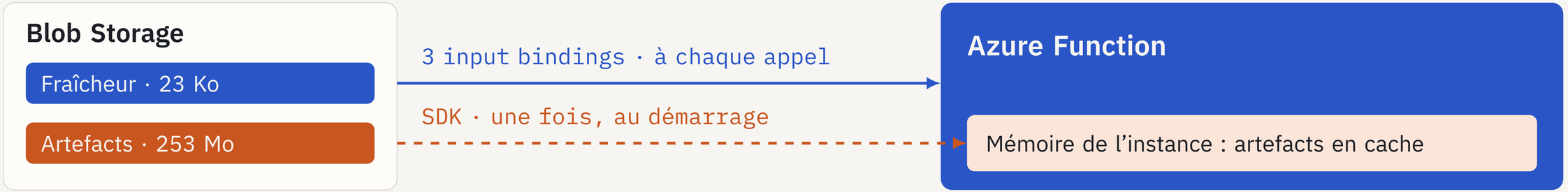
Un seul service : la Function lit Blob Storage elle-même.

Architecture 2 : gratuite sans être bridée

Critère	Architecture 1 · avec API	Architecture 2 · sans API
Services à maintenir	Deux : l'API et la Function	Un : la Function
Modèle de coût	Forfait de l'API, plus l'usage	Usage seul
Version gratuite	App Service F1 : 60 min de CPU par jour, 1 Go de RAM, sans SLA	Quota Flex Consumption : 250 000 exécutions et 100 000 Go-s par mois
Au-delà du gratuit	Forfait mensuel, même sans trafic	Proportionnel au trafic
Latence	Un saut réseau de plus	Prédiction dans la Function
Quand la choisir	Trafic régulier, plusieurs applications clientes	MVP, trafic faible et irrégulier

UN ÉCART À LA RECOMMANDATION, MESURÉ

253 Mo à chaque appel, ou une seule fois



Tout par le binding, comme recommandé



8 s / appel

Binding pour la fraîcheur, SDK pour les modèles · retenu



0,2 s / appel

artefacts, 253 Mo · fraîcheur, 23 Ko, relue à chaque appel · calcul de la recommandation

Ricochet · 22 / 25 · Chaque bloc = un appel ; largeurs indicatives

POURQUOI CETTE ARCHITECTURE · LE COÛT

De la gratuité découle toute la chaîne

Pas de serveur permanent → **Plan à la consommation** dans le quota gratuit → **Aucune instance « always ready »** → **Démarrage à froid, accepté en démo** → **Artefacts en cache** 0,2 s par appel

COÛT RÉEL FACTURÉ

0 €

Du 01/08/2026 au 06/11/2026, relevé dans Azure Cost Management, Function, stockage et télémétrie compris.

En production, le premier appel après une mise en veille prend **≈ 8 s**. Leviers : artefacts allégés à ≈ 110 Mo (sans ALS ni SVD), instance « always ready » (payante), ou passage à l'architecture 1.

CE QUI SÉPARE UN SERVICE D'UNE DÉMONSTRATION

Quatre défaillances sans la moindre erreur

Dérive d'artefact
Le cache est jugé sur la taille et la date du blob.

Catalogue incohérent
Tout identifiant hors catalogue est filtré.

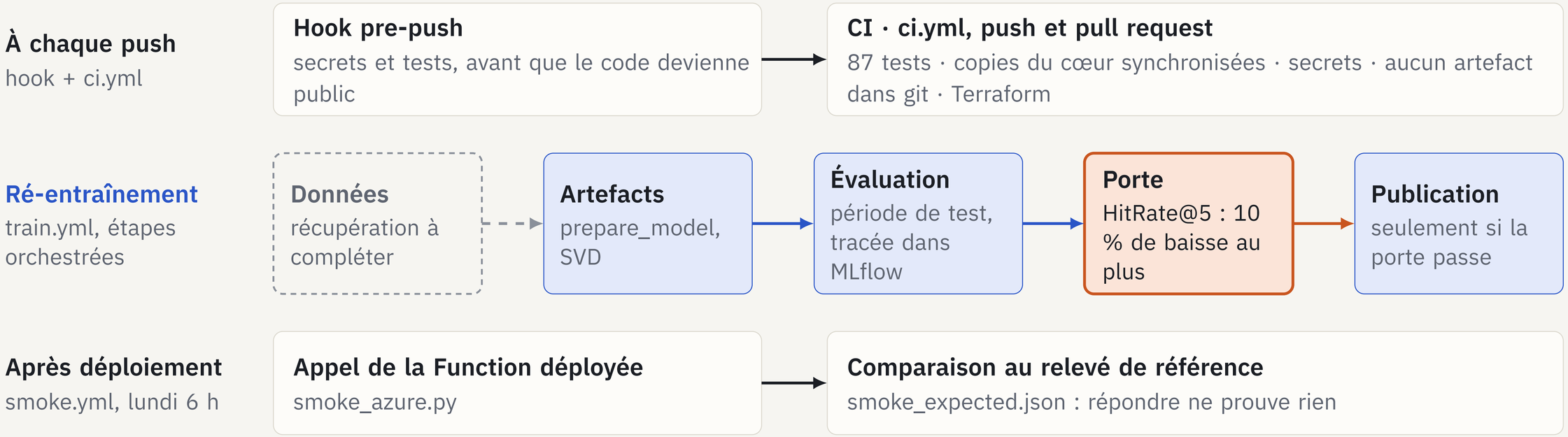
Cache périmé
Le conteneur est énuméré, pas lu dans une liste figée.

Requêtes concurrentes
L'historique d'un appel ne fuit pas vers un autre.

Le service répondait 200 avec de mauvaises recommandations. Les deux premières ont été vues en comparant au calcul local.

MLOPS · GITHUB ACTIONS COMME ORCHESTRATEUR

Trois chaînes, une porte de qualité



Côté service, pas d'orchestrateur : chaque Function a son trigger, elles communiquent par le stockage. L'orchestration est là où il y a des étapes à enchaîner : le ré-entraînement.